

[haskell] Haskell Day-Zero

1 message

deepak ravi <deepak.ravi@gmail.com>

Wed, 23 Jan, 2008 at 10:46 pm

To: krishnair1@gmail.com, mathewcfrancis@gmail.com

Hi hari/math,

This is a quick summary on what we did on day-zero:
would appreciate if you could send me any comments/corrections.

Haskell Day-Zero

=====

1. Problem Solving

1. What we have

1. We have an idea in our mind(some algorithm+strategy)
2. The Hardware understands Machine language

2. How to

1. Represent the idea in a formal language(intermediate language or programming high levellanguage, say haskell)
Note: the language chosen should be more closer to our idea in mind than machine language
2. Get a translator which understands both this formal language and machine language (eg: ghc which can translate haskell to machine language)

What if idea in our mind == Mathematical functions/expressions

ie. How to represent a mathematical function/expression in a formal language/haskell

Explained through examples:

Our aim is to get familiarised with haskell language and its way of thinking, not on language features, and its technical details.

i.e get familiarised with

1. layout, syntax, lexicals, etc.
2. expressions

Representation of function: $f(x)$ is $f\ x$.

$f\ x$ is same as $(f\ x)$

the first one is the function to be called and the rest are arguments

$f(x,y,z)$ is $f\ x\ y\ z$, is same as $(f\ x\ y\ z)$

Example 1

Maths: $f(x)=3$

Haskell: $f\ x = 3$

Example 2

Maths: $g(x) = f(x) + f(x)$

Haskell: $g\ x = f\ x + f\ x$

$g\ x = (f\ x) + (f\ x)$

Example 3

Maths: $g(x) = f(x-1)$

Haskell: $g(x) = f\ (x-1)$

Example 4

Maths: $h(x) = f.g(x)$

Haskell: $h\ x = f . g\ x$,

Remember: $'.'$ is the function composition operator in maths. ie. $f.g(x) = f(g(x))$

Example 5

Maths: $h = f.g$

Haskell: $h = f.g$

Example 6

Maths: $f(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{otherwise} \end{cases}$

Haskell: $f\ x \mid x > 0 = 1$

| otherwise =0

Please note the layout and how "| x>0" relates to "if x>0" etc.

Example 7

Maths: $\text{fact}(x) = \begin{cases} 1, & \text{if } x=0 \\ x \cdot \text{fact}(x-1), & \text{if } x>0 \end{cases}$

Haskell: $\text{fact } x \mid x==0 = 1$
 $\mid x>0 = x * \text{fact } (x-1)$

Please note that in maths, = operator has got two meanings, x=0, means assign 0 to x
if(x=0) means compare whether x and 0 are same.

to avoid this ambiguity, in haskell, we separate both the concept. by using two different operators,
= and ==.

x=0 means assign 0 to x

x==0 means compare whether x and 0 are same.

Also note the recursion:

Recursion and "divide and conquer"

In recursion,

1. we break the problems into sub problems.
2. tell how to solve the main problem if all the sub-problems are solved.

Example 8:

Maths: $\text{signum}(x) = \begin{cases} 1, & \text{if } x>0 \\ -1, & \text{if } x<0 \\ 0, & \text{otherwise} \end{cases}$

Haskell: $\text{signum } x \mid x>0 = 1$
 $\mid x<0 = -1$
 $\mid x==0 = 0$

Example 9 (written by hari)

Maths: $\sin(x) = \begin{cases} 0 & \text{if } x=0, \\ 1 & \text{if } x=90 \end{cases}$

Haskell: $\sin x \mid x==0 = 0$
 $\mid x==90 = 1$

Example 10: Pair, tuple, vectors

Maths: (x,y) ==> Ordered Pair

Haskell: (x,y) -- ordered pair

Example 11: solving roots of quadratic eqn

Haskell:

roots a b c | det >= 0 = (r1,r2)
where r1 = (-b + d)/a2
r2 = (-b - d)/a2
d = sqrt det
det= b * b - 4*a*c

Example 12: Write a function to get first element of a pair(x,y)

fst (x,y) = x

snd (x,y) = y

Lists / Sequence

=====

Examples: [1,2,3,4,5,6,7]

list of prime numbers < 20 ==> [1,2,3,5,7,11,13,17,19]

and so on

Definition of list:

List is either

empty ie. []

or

an element and another list, ie. x : Another List

[1,2,3,4,5] = 1:2:3:4:5:[] = 1:[2,3,4,5] etc.

Note the definition of list itself is recursive.

Example: sum the elements of list

$\text{sum } [] = 0$
 $\text{sum } (x:xs) = x + \text{sum } xs$

Note: $(x:xs)$ is in brackets because of $:$ operator

Example: length of list, written by hari

$\text{length } [] = 0$
 $\text{length } (x:xs) = 1 + \text{length } xs$

Another way of representing lists

Example 1:

Note: In maths, sometimes we define a sequence like this.

$\text{even} = [x \mid x < N, x \text{ remainder } 2 \text{ is } 0]$

in haskell, there is a similar notation too.

$\text{even} = [x \mid x < n, x \text{ `rem` } 2 = 0]$

Example 2:

What if in maths we define even as follows:

$\text{even} = [2x \mid x < N]$
Haskell: $[2*x \mid x < n]$

Example 3:

Well, how to define natural numbers N

In maths, $N=1,2,3,4,5,6\dots$

In Haskell: $n=[1..]$

Example 4:

Maths: $\text{crossproduct}(A,B) = \{ (x,y) \mid x < A, y < B \}$

Haskell: $\text{crossproduct } a \ b = [(x,y) \mid x < a, y < b]$

ie. $A=[1,2,3], B=[a,b,c],$
 $\text{crossproduct } A \ B = [(1,a) \dots]$

Small discussion on:

1. recursive way of thinking
2. once mastered we won't be writing recursive functions!!!!
3. we will be finding patterns and replace them with functions
4. higher order functions, and so

Idea suggested by hari: foldl

generic implementation of product and sum

$\text{product } d [] = d$
 $\text{product } d (x:xs) = \text{product } (x*d) \ xs$

a bit of generalizing

$\text{mul } a \ b = a*b$
 $\text{product } d [] = d$
 $\text{product } d (x:xs) = \text{product } (\text{mul } d \ x) \ xs$

a bit of generalizing, this is what hari was suggesting, well it's called foldl

$\text{foldl } f \ d [] = d$
 $\text{foldl } f \ d (x:xs) = \text{foldl } f \ (f \ d \ x) \ xs$

Note: just introduce f. replace mul with f
Note: foldl takes f as argument

Yet another abstraction: map

$[1,2,3,4] \Rightarrow [10,20,30,40]$
 $[1,2,3,4\dots] \Rightarrow [2,4,6,8,\dots]$

given a list, generate a new list with each element x replaced by $f(x)$

$[a,b,c] \Rightarrow [f\ a, f\ b, f\ c]$

generalize this idea.

$\text{map } f [] = []$
 $\text{map } f (x:xs) = f\ x : \text{map } f\ xs$

A small application of map.

Given a list of integers represented in strings,
convert each of string to integer, increment them, and
generate a new list of integers in string representation form

$f = \text{map } (\text{intToString} . \text{inc} . \text{stringToInt})$
Note: In haskell intToString is show, and stringToInt is read

Also, think how we will implement this problem in C++, Java, etc.
or write it in algorithmic notation, so and so..

End of Day-Zero :)

--
Regards,
Deepak Ravi

PS: Please follow net-etiquette